1. Title Page

Project Title:
The Cursed Kingdom: A Console-Based Adventure Game Using Object-Oriented Programming in C++

---

2. Abstract

This project presents **The Cursed Kingdom**, a console-based adventure game developed in C++ utilizing Object-Oriented Programming (OOP) principles. The game is designed to provide two distinct gameplay modes: Sandbox Mode, which allows players to experiment freely with inventory and combat mechanics, and Story Mode, which offers a structured narrative with turn-based battles against enemies. Core functionalities include a user authentication system, dynamic inventory management, interactive ASCII art for visual engagement, and a combat system emphasizing strategic decisions between attack and defense. The primary purpose of this project is to demonstrate practical implementation of OOP concepts such as classes, encapsulation, inheritance, polymorphism, and file handling in a cohesive, functional software application.

---

3. Introduction

Games have long been a compelling medium for exploring programming concepts, as they naturally require the integration of multiple functionalities such as user interaction, state management, and dynamic behavior. With the rise of Object-Oriented Programming, designing games using OOP concepts has become a standard approach to managing complexity and enhancing maintainability. The motivation behind developing **Cursed Kingdom** was twofold: to strengthen my understanding of core C++ OOP techniques and to create an engaging, interactive game that showcases these skills.
The choice of a console-based text adventure was deliberate, allowing focus on logic and design rather than graphics. The game challenges players to navigate different modes, manage resources, and engage in combat through intuitive commands and visual ASCII art. This project bridges the gap between academic concepts and real-world application, serving both educational and entertainment purposes.

---

4. Project Objectives

The goals and objectives of this project are outlined as follows:

- **Design and implement a fully functional text-based game using C++**
  Develop a program that runs entirely in the console with a user-friendly interface and smooth interaction.
- **Utilize Object-Oriented Programming concepts effectively**
  Structure the project with classes and objects representing players, enemies, inventory items, and game logic to illustrate encapsulation, inheritance, and polymorphism.
- **Develop a secure and simple user authentication system**
  Allow players to sign up and log in using persistent storage, demonstrating file I/O operations.
- **Create two distinct game modes**
  Implement Sandbox Mode for free experimentation with items and Story Mode for narrative-driven challenges with enemies and turn-based combat.
- **Implement a dynamic inventory system**
  Allow addition and display of items like weapons and potions, and provide meaningful interaction with these items during gameplay.
- **Enhance user experience with ASCII art and color-coded console output**
  Use simple graphics to represent items, enemies, and game elements, making the text interface more engaging.
- **Demonstrate robust input validation and error handling**
  Ensure the program handles invalid inputs gracefully, enhancing reliability.

---

5. Scope of the Project

This project is designed with clear boundaries to focus on core programming skills while delivering a complete gaming experience.

In Scope:

- Console-based user interface with text input and output
- User account management with login and signup functionality
- Two gameplay modes: Sandbox and Story, each with specific features
- Turn-based combat system with attack and defense options
- Inventory management including weapons, healing items, and armor
- Use of ASCII art to visually represent game elements
- Implementation of fundamental OOP concepts (classes, encapsulation, inheritance)
- Input validation and error handling to improve usability

Out of Scope:

- Graphical User Interface (GUI) or 3D graphics
- Networked multiplayer or online features
- Complex AI opponents or machine learning elements
- Save/load game progress beyond user login data
- Sound effects or music integration
- Extensive storyline beyond basic narrative framework in Story Mode

This clear scoping ensures the project remains focused on the technical aspects of OOP while maintaining an enjoyable gameplay experience.

---

6. Methodology

The development of **The Cursed Kingdom** follows a methodical approach centered on applying OOP principles in C++.

- **Object-Oriented Design:** The game structure is broken down into logical entities such as User, Player, Enemy, Item, Inventory, and GameManager. Each class encapsulates related data and behavior, ensuring modularity and separation of concerns.
- **Encapsulation:** Private data members are accessed through public methods, safeguarding internal state and enabling controlled interaction. For example, the

Inventory class manages a collection of Item objects, providing methods to add or display items.

- **Inheritance and Polymorphism:** While the current implementation has distinct enemy types (e.g., Goblin, Orc, Cursed King), the design allows extension through inheritance to create specialized enemy classes. Polymorphism would enable dynamic behavior changes in future versions.
- **User Input Handling and Validation:** To ensure smooth gameplay, all user inputs undergo rigorous validation using C++ stream state checking (cin.fail(), cin.clear()) and input range enforcement.
- **File Handling for User Authentication:** Player credentials are securely stored and retrieved using file I/O streams, allowing persistent user data between sessions.
- **Game Flow Management:** The main game loop orchestrates login/signup, mode selection, and gameplay, relying on helper functions like startSandboxMode(), fightEnemy(), and showSandboxHints() for modular code structure.
- **Visual Engagement via ASCII Art:** Manually crafted ASCII art adds immersive visual cues to gameplay events, such as weapon acquisition and enemy encounters, enhanced with ANSI color codes for better clarity.

This methodology ensures that the game not only functions correctly but is also maintainable, extensible, and educational in demonstrating OOP.

---

7. Resources and Tools

The following resources and tools were utilized during the development of this project:

- **Programming Language:** C++ (Standard C++11/14 features)
- **Development Environment:** Visual Studio Code with the Code Runner extension for compilation and execution.
- **Libraries:** Standard Template Library (STL) for basic containers such as vectors (for inventory management).
- **Text Formatting:** ANSI escape codes for colored console output.
- **ASCII Art:** Created manually using online ASCII art generators and text editors.

- **Reference Material:**
    - C++ official documentation
    - Online tutorials and articles on OOP concepts
    - ChatGPT and online programming forums for troubleshooting and best practices

These tools and resources supported effective development and debugging throughout the project lifecycle.

---

8. Challenges and Solutions

Challenge 1: Input Validation and User Experience

Problem: Ensuring the user inputs valid commands, numeric ranges, and choices without crashing the program.
Solution: Implemented robust input validation loops using cin.fail(), clearing input buffers, and prompting users repeatedly until valid input is received. This improved reliability and user satisfaction.

Challenge 2: Managing Complex Game Logic

Problem: The combat system required multiple states (attack, defense, healing, armor choices) within limited turns and retries.
Solution: Used nested loops and boolean flags to track player choices and states per turn. Modularized the logic into helper functions such as fightEnemy() to maintain clarity.

Challenge 3: Balancing Game Difficulty and Fairness

Problem: Designing the conditions for winning or losing to feel fair and engaging without being too easy or too punishing.
Solution: Introduced attack thresholds, healing requirements, and armor effects, encouraging strategic decisions by the player. Playtesting was used to tune these parameters.

Challenge 4: Integrating ASCII Art without Cluttering the Code

Problem: Including large ASCII art in code made it hard to read and maintain.
Solution: Separated ASCII art into dedicated functions (e.g., displayGoblinArt()) and used string literals with color codes to keep the main logic clean.

Challenge 5: File Handling for User Authentication

Problem: Storing and retrieving user credentials securely and efficiently.
Solution: Used simple file streams to save usernames and passwords with basic checks. While not encrypted (outside the project scope), it demonstrated persistent storage and file I/O skills.

---

9. Project Timeline

Week	Milestone	Description
Week 1	Project Planning and Research	Defined project scope, objectives, and gathered references.
Week 2	Class Design and Setup	Created core classes like User, Player, Inventory, and Item.
Week 3	User Authentication Module	Implemented login and signup with file storage.
Week 4	Inventory System and Item Management	Designed dynamic inventory with add/show functionality.
Week 5	Sandbox Mode Implementation	Developed free-play mode with item additions and hints.
Week 6	Story Mode and Combat System	Built turn-based fight mechanics with enemies and defense choices.
Week 7	Input Validation and ASCII Art Integration	Added robust input checks and immersive ASCII visuals.
Week 8	Testing, Debugging, and Documentation	Finalized code, fixed bugs, and prepared documentation.

---

10. Conclusion

This project successfully demonstrates how foundational Object-Oriented Programming concepts can be leveraged to create an interactive and enjoyable console game in C++. The design emphasizes clean code organization, encapsulation, modularity, and user interaction management. While the game remains text-based, the use of ASCII art and colorful output enhances engagement and provides visual interest despite the limitations of the console interface.

The project outcomes include a working login system, dual gameplay modes, and a flexible combat and inventory system that can be extended in future iterations. For further enhancements, the game could integrate GUI frameworks, save/load game progress, introduce AI-controlled enemies, or expand the narrative with richer storytelling elements.

Overall, "The Cursed Kingdom" served as an effective educational exercise, reinforcing my programming skills, problem-solving abilities, and understanding of C++ OOP principles while delivering a polished, playable application.